

Testing de Software 2024
Práctico 8

Github assignment: https://classroom.github.com/a/F_pnGRCr

Ejercicio 1: Considere la implementación de `assignment7_exercises.nclNodeCachingLinkedList`. Genere Test con Randoop para esta clase. Analizar la salida obtenida. Corra los test generados.

- En el caso de tener algún `errorTest`, analizarlo detalladamente, realizar debbuging para encontrar el defecto. Corregir los defectos encontrados.
- En el caso de haber encontrado defectos, luego de corregirlos, vuelva a generar test, hasta que obtenga solo una suite de test de regresión.
- Proveer a Randoop contratos extras: Invariante de representación (`repOK`), vuelva a generar test y corrija defectos en el caso de encontrarlos.
- Medir cobertura de branches y de mutación de la suite de test de regresión obtenida.

Nota: En el repositorio se encuentra un script `gen-test-randoop.sh` que puede ser usado para correr randoop. El mismo solo hace uso de los parámetros básicos, debe ser analizado. Puede modificarlo si lo desea. Se asume la estructura de directorio: “`src/test/java`” para los test y “`target/classes`” para los `.class`. Esto puede ser modificado también según se requiera. Por otra parte, necesitará agregar randoop como dependencia local maven (ver: `mvn install:install-file`)

Ejercicio 2: Analizar el código fuente dado en la clase `fileExample` de `assignment8_exercises.fileContents`. Generar test con Evosuite y con Randoop para dicha clase. Analizar y Explicar el resultado obtenido en ambos casos.

Ejercicio 3. Investigue que mecanismos provee Randoop para mejorar la generación en escenarios como en el del ejercicio 2

Ejercicio 4. La clase `IPBlackList` (`assignment8_exercises.logging`) posee un método `login` que se comporta según las siguientes reglas:

- almacena el último IP que intento loguearse usando `LoginService`.
- almacena el número de intentos fallidos consecutivos del mismo IP.

-si un mismo IP falla tres veces consecutivas al intentar loguearse, esta IP se almacena en una lista negra.

Realice los siguientes test:

- Chequee que después de 3 intentos fallidos con el mismo IP, IP figura en la lista negra.
- Chequee que, si se intenta loguearse menos de 3 veces, el IP no figura en la lista negra.

Ejercicio 5: Se provee una implementación del Sistema **fail2ban** el cual permite el acceso a un servidor cuando se sospecha que quien lo accede puede estar efectuando un ataque por fuerza bruta. El sistema mantiene, básicamente, los siguientes datos:

- bans: Una lista de direcciones IP prohibidas (banned addresses). si una conexión proviene de una de estas direcciones, se rechaza. Las IPs están prohibidas solo por un período de tiempo.
- exceptions: Un conjunto de excepciones: direcciones IP que corresponden a hosts conocidos y confiables, y de los cuales nunca se rechazan pedidos de conexión.
- lastUpdate: El último tiempo de actualización de la bans

La lista bans (implementada como una lista simplemente encadenada con elemento centinela) almacena objetos IPBan, cada uno de los cuales está compuesto por el IP del host y el tiempo de expiración de la prohibición (expires). Esta lista está estrictamente (no admite repetidos) ordenada con respecto al tiempo de expiración de cada IP. Además, esta lista tampoco admite IPs repetidos.

lastUpdate almacena el tiempo de la última actualización de la lista bans, por lo tanto, ningún IP baneado puede tener un tiempo de expiración menor a lastupdate. Cada vez que se actualiza la lista bans (método update), se eliminan todas las entradas de esta lista con tiempo de expiración menor o igual que el tiempo actual del sistema.

Por otro lado, exceptions almacena IPs de manera no repetida y fue implementado con una lista simplemente encadenada con elemento centinela.

Por último, exceptions y bans no comparten elementos, es decir, no es posible encontrar un IP prohibido que pertenezca al conjunto de IP confiables.

- a) Complete la clase Server, con una implementación de la rutina repOK(), que capture el invariante de representación de la clase según las restricciones arriba especificadas.
- b) Corra Randoop con un tiempo máximo de 30 seg. para crear una suite de test que ejercite los métodos de la clase Server presentada en el ejercicio anterior (utilice el repOK ya implementado). Si encontró fallas, reporte las mismas y debuggee para

encontrar el defecto. Evalúe Branch Coverage y habilidad para matar mutantes de su suite de test.

- c) Corra Evosuite con un tiempo máximo de 30 seg para crear una suite de test que ejercite los métodos de la clase Server. Si aún encontró fallas, reportar las mismas y debuggee para encontrar el defecto. Realice una comparación con randoop con respecto a la cobertura de branches y de mutación obtenida.
- d) ¿Es posible utilizar repOK() para depurar su implementación con Evosuite? ¿Como?
- e) Construya una propiedad (usando jqwik) sobre método update. Para ello deberá construir 1 generador de parámetros.